

Search-to-decision reducibility for NP languages

María Alejandra Schild

Pontificia Universidad Católica de Chile
Computational Complexity Seminar

April 17, 2026

- 1 The usual story
- 2 Search-to-decision without SAT oracles
- 3 An NP language which is (probably) not search-to-decision reducible

Certifying positive answers with a SAT oracle

Suppose we have an oracle for deciding CNF-SAT, but no one believes us.

Can we use our oracle to construct a polynomial-time machine that, given a CNF formula φ , outputs the correct answer as well as a *witness*?

Certifying positive answers with a SAT oracle

Suppose we have an oracle for deciding CNF-SAT, but no one believes us.

Can we use our oracle to construct a polynomial-time machine that, given a CNF formula φ , outputs the correct answer as well as a *witness*?

By “witness” we mean a certificate of size $\text{poly}(|\varphi|)$ that can be given to a polynomial-time verifier. The verifier receives as inputs the formula φ and the claimed witness.

Certifying positive answers with a SAT oracle

Suppose we have an oracle for deciding CNF-SAT, but no one believes us.

Can we use our oracle to construct a polynomial-time machine that, given a CNF formula φ , outputs the correct answer as well as a *witness*?

By “witness” we mean a certificate of size $\text{poly}(|\varphi|)$ that can be given to a polynomial-time verifier. The verifier receives as inputs the formula φ and the claimed witness.

Unless $\text{NP} = \text{coNP}$, we do not expect witnesses for **negative instances** (in general).

For **positive instances**, witnesses exist and **we can use our oracle to construct them**.

Substitution by constants

$\top$ (True) and $\perp$ (False) are 0-ary connectives (*logical constants*).

Recall that $\neg\top = \perp$ and $\neg\perp = \top$.

Substitution by constants

$\top$ (True) and $\perp$ (False) are 0-ary connectives (*logical constants*).

Recall that $\neg\top = \perp$ and $\neg\perp = \top$.

Let $\varphi(x_1, \dots, x_n)$ be a CNF formula and let $b \in \{\top, \perp\}$.

We write $\varphi[x_i \leftarrow b]$ to denote the formula obtained from φ by replacing every occurrence of the literal x_i by the constant b and every occurrence of the literal $\neg x_i$ by $\neg b$, and then applying the following rules:

- any clause containing $\top$ can be deleted;
- any occurrence of $\perp$ inside a clause can be deleted;
- if a clause becomes empty, then the whole formula turns into $\perp$;
- if all clauses disappear, then the whole formula turns into $\top$.

Substitution by constants

$\top$ (True) and $\perp$ (False) are 0-ary connectives (*logical constants*).

Recall that $\neg\top = \perp$ and $\neg\perp = \top$.

Let $\varphi(x_1, \dots, x_n)$ be a CNF formula and let $b \in \{\top, \perp\}$.

We write $\varphi[x_i \leftarrow b]$ to denote the formula obtained from φ by replacing every occurrence of the literal x_i by the constant b and every occurrence of the literal $\neg x_i$ by $\neg b$, and then applying the following rules:

- any clause containing $\top$ can be deleted;
- any occurrence of $\perp$ inside a clause can be deleted;
- if a clause becomes empty, then the whole formula turns into $\perp$;
- if all clauses disappear, then the whole formula turns into $\top$.

Note that either $\varphi[x_i \leftarrow b]$ is a constant or a CNF formula that contains no constants and no occurrence of x_i .

SAT is self-reducible

The following function receives as input a CNF formula φ and decides whether it is satisfiable.

```
1: function SAT( $\varphi$ )
2:   if  $\varphi = \top$  then
3:     return True
4:   else if  $\varphi = \perp$  then
5:     return False
6:   else
7:     let  $x$  be a propositional variable that appears in  $\varphi$ 
8:     return SAT( $\varphi[x \leftarrow \top]$ )  $\vee$  SAT( $\varphi[x \leftarrow \perp]$ )
```

Computing witnesses for positive instances of SAT

The following algorithm receives as input a CNF formula $\varphi(x_1, \dots, x_n)$. The output is either a satisfying assignment of φ or FAIL, if no such assignment exists.

```
1: if  $\neg \text{SAT}(\varphi)$  then  
2:   return FAIL  
3:  $\sigma \leftarrow \{\top\}^n$   
4: for  $i \leftarrow n, n-1, \dots, 1$  do  
5:   if  $\text{SAT}(\varphi[x_i \leftarrow \top])$  then  
6:      $\varphi \leftarrow \varphi[x_i \leftarrow \top]$   
7:   else  
8:      $\sigma_i \leftarrow \perp$   
9:      $\varphi \leftarrow \varphi[x_i \leftarrow \perp]$   
10: return  $\sigma$ 
```

Computing witnesses for positive instances of SAT

The following algorithm receives as input a CNF formula $\varphi(x_1, \dots, x_n)$. The output is either a satisfying assignment of φ or FAIL, if no such assignment exists.

```
1: if  $\neg \text{SAT}(\varphi)$  then  
2:   return FAIL  
3:  $\sigma \leftarrow \{\top\}^n$   
4: for  $i \leftarrow n, n-1, \dots, 1$  do  
5:   if  $\text{SAT}(\varphi[x_i \leftarrow \top])$  then  
6:      $\varphi \leftarrow \varphi[x_i \leftarrow \top]$   
7:   else  
8:      $\sigma_i \leftarrow \perp$   
9:      $\varphi \leftarrow \varphi[x_i \leftarrow \perp]$   
10: return  $\sigma$ 
```

Given access to a SAT oracle, this algorithm runs in polynomial time.

Existence of witnesses for all NP languages

A language $L \subseteq \{0, 1\}^*$ is in NP if and only if there exists a polynomial $p(\cdot)$ and a polynomial-time Turing machine V such that, for all $x \in \{0, 1\}^*$,

$$x \in L \iff \exists y \in \{0, 1\}^{p(|x|)} V(x, y) = \text{True}.$$

Existence of witnesses for all NP languages

A language $L \subseteq \{0, 1\}^*$ is in NP if and only if there exists a polynomial $p(\cdot)$ and a polynomial-time Turing machine V such that, for all $x \in \{0, 1\}^*$,

$$x \in L \iff \exists y \in \{0, 1\}^{p(|x|)} V(x, y) = \text{True}.$$

We say that V is a *p-verifier* for L . Note that we can just omit the polynomial $p(\cdot)$ by forcing V to reject all pairs (x, y) such that $|y| \neq p(|x|)$.

Existence of witnesses for all NP languages

A language $L \subseteq \{0, 1\}^*$ is in NP if and only if there exists a polynomial $p(\cdot)$ and a polynomial-time Turing machine V such that, for all $x \in \{0, 1\}^*$,

$$x \in L \iff \exists y \in \{0, 1\}^{p(|x|)} V(x, y) = \text{True}.$$

We say that V is a *p-verifier* for L . Note that we can just omit the polynomial $p(\cdot)$ by forcing V to reject all pairs (x, y) such that $|y| \neq p(|x|)$.

Idea of the proof.

($\Leftarrow$) Use nondeterminism to guess a witness $y \in \{0, 1\}^{p(|x|)}$.

Existence of witnesses for all NP languages

A language $L \subseteq \{0, 1\}^*$ is in NP if and only if there exists a polynomial $p(\cdot)$ and a polynomial-time Turing machine V such that, for all $x \in \{0, 1\}^*$,

$$x \in L \iff \exists y \in \{0, 1\}^{p(|x|)} V(x, y) = \text{True}.$$

We say that V is a p -*verifier* for L . Note that we can just omit the polynomial $p(\cdot)$ by forcing V to reject all pairs (x, y) such that $|y| \neq p(|x|)$.

Idea of the proof.

- $(\Leftarrow)$ Use nondeterminism to guess a witness $y \in \{0, 1\}^{p(|x|)}$.
- $(\Rightarrow)$ Let M be a polynomial-time, nondeterministic Turing machine that decides L . Take $p(\cdot)$ as a polynomial that bounds the running time of M , and consider V that, on input (x, y) , deterministically simulates M on input x by using y to make the nondeterministic choices.



Levin reductions

Consider $L_1, L_2 \in \text{NP}$ and let V_1 and V_2 be p-verifiers for L_1 and L_2 , resp.

Levin reductions

Consider $L_1, L_2 \in \text{NP}$ and let V_1 and V_2 be p-verifiers for L_1 and L_2 , resp.

A *Levin reduction* from the pair (L_1, V_1) to the pair (L_2, V_2) is a pair of polynomial-time computable functions (f, g) such that:

- For every $x \in \{0, 1\}^*$,

$$x \in L_1 \iff f(x) \in L_2.$$

Levin reductions

Consider $L_1, L_2 \in \text{NP}$ and let V_1 and V_2 be p-verifiers for L_1 and L_2 , resp.

A *Levin reduction* from the pair (L_1, V_1) to the pair (L_2, V_2) is a pair of polynomial-time computable functions (f, g) such that:

- For every $x \in \{0, 1\}^*$,

$$x \in L_1 \iff f(x) \in L_2.$$

- For every $x \in \{0, 1\}^*$ and $y \in \{0, 1\}^*$,

$$V_2(f(x), y) = \text{True} \implies V_1(x, g(x, y)) = \text{True}.$$

Cook-Levin theorem for Levin reductions

Let V_{eval} be the p-verifier for SAT that, for every propositional formula φ and $y \in \{0, 1\}^{|\varphi|}$, $V_{\text{eval}}(\varphi, y)$ is the truth value of evaluating the variables of φ according to the first n bits of y , where n is the number of variables appearing in φ .

Cook-Levin theorem for Levin reductions

Let V_{eval} be the p-verifier for SAT that, for every propositional formula φ and $y \in \{0, 1\}^{|\varphi|}$, $V_{\text{eval}}(\varphi, y)$ is the truth value of evaluating the variables of φ according to the first n bits of y , where n is the number of variables appearing in φ .

Theorem (Cook-Levin)

Consider $L \in \text{NP}$ and let V be a p-verifier for L . Then there exists a Levin reduction from (L, V) to $(\text{SAT}, V_{\text{eval}})$.

Cook-Levin theorem for Levin reductions

Let V_{eval} be the p-verifier for SAT that, for every propositional formula φ and $y \in \{0, 1\}^{|\varphi|}$, $V_{\text{eval}}(\varphi, y)$ is the truth value of evaluating the variables of φ according to the first n bits of y , where n is the number of variables appearing in φ .

Theorem (Cook-Levin)

Consider $L \in \text{NP}$ and let V be a p-verifier for L . Then there exists a Levin reduction from (L, V) to $(\text{SAT}, V_{\text{eval}})$.

Corollary

Consider $L \in \text{NP}$ and let V be a p-verifier for L . Then there exists a polynomial-time machine M with access to a SAT oracle such that:

- If $x \in L$, then $V(x, M(x)) = \text{True}$.
- If $x \in \{0, 1\}^* \setminus L$, then $M(x)$ outputs FAIL.

Is that all?

Computing witnesses with weaker oracles

Consider the language GI of pairs (G, H) of isomorphic graphs. This problem is in NP, and isomorphisms are the natural witnesses.

Computing witnesses with weaker oracles

Consider the language GI of pairs (G, H) of isomorphic graphs. This problem is in NP, and isomorphisms are the natural witnesses.

Of course, if we have access to a SAT oracle, we would be able to construct an isomorphism between G and H as long as the graphs are actually isomorphic. However, that would be overkill.

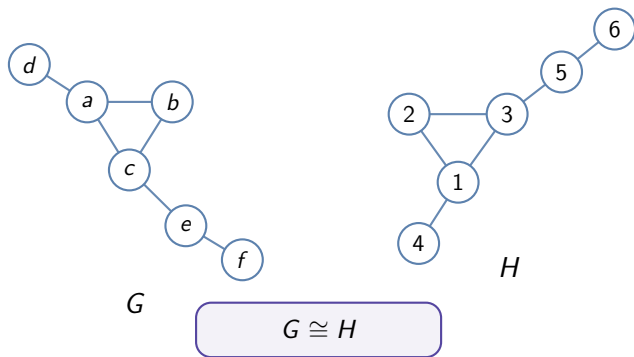
Computing witnesses with weaker oracles

Consider the language GI of pairs (G, H) of isomorphic graphs. This problem is in NP, and isomorphisms are the natural witnesses.

Of course, if we have access to a SAT oracle, we would be able to construct an isomorphism between G and H as long as the graphs are actually isomorphic. **However, that would be overkill.**

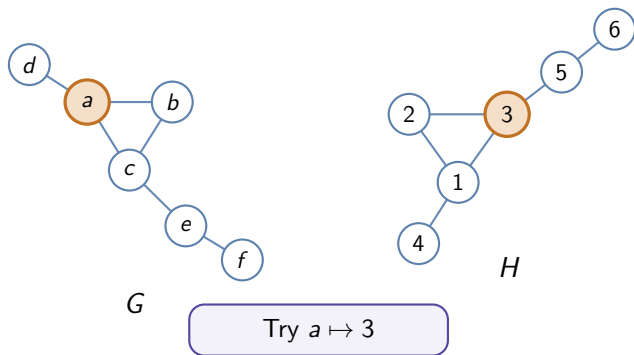
What if we only have access to an oracle for the language GI?

A pair of isomorphic graphs



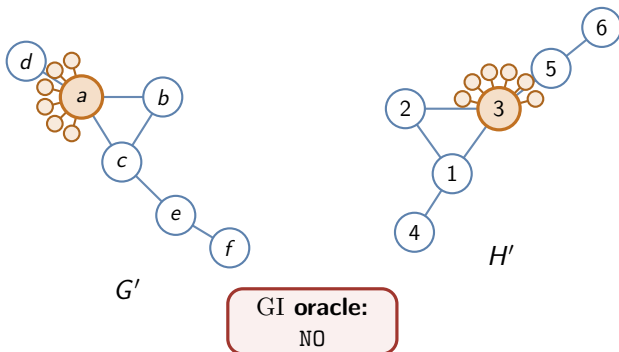
We know an isomorphism exists because the oracle told us.
Now we want to actually construct an isomorphism

First try: a bad pin



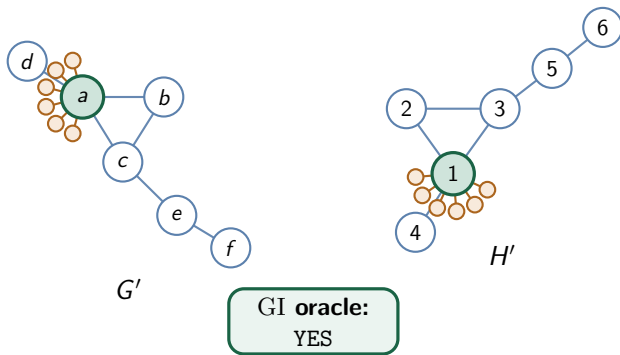
Are the graphs isomorphic if we force the colored vertices to match?

Replace the color by a 6-leaf gadget



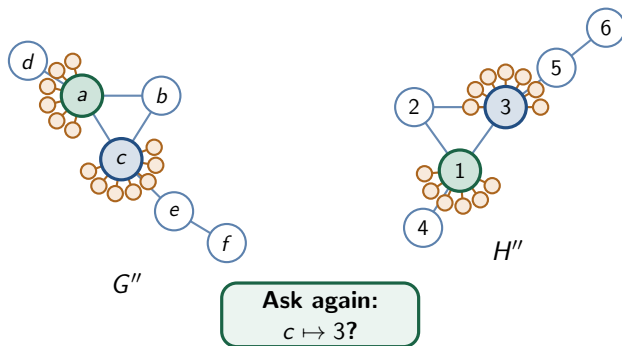
So the pin $a \mapsto 3$ cannot be extended to an isomorphism.

Now choose the right first pin



We keep these gadgets to maintain the first pair of the isomorphism.

We keep the first pin and add a new one



The first pair still has the gadgets with 6 leaves.
For the second pair we must use gadgets with 7 leaves.

Computing isomorphisms in polynomial time

If the oracle answers NO, we just need to keep trying.

Because we know that at least one isomorphism exists, at least one pair must work.

In each step we only need to try a linear number of pairs.

This gives us a polynomial-time algorithm for constructing isomorphisms given access to an oracle for GI.

In general, given access to an oracle for $L \in \text{NP}$, can we compute witnesses for positive instances of L in polynomial time?

Is this question even well-defined?

A problem with two different p-verifiers

Consider the following language of quadratic residues (with inputs in binary):

$$\text{QR} = \left\{ (a, N) \in \mathbb{Z}_{>0}^2 \mid \exists x \in \mathbb{N} \ a \equiv_N x^2 \right\},$$

where $a \equiv_m b$ means that m divides $a - b$.

A problem with two different p-verifiers

Consider the following language of quadratic residues (with inputs in binary):

$$\text{QR} = \left\{ (a, N) \in \mathbb{Z}_{>0}^2 \mid \exists x \in \mathbb{N} \ a \equiv_N x^2 \right\},$$

where $a \equiv_m b$ means that m divides $a - b$.

For simplicity, we add the following (polynomial-time decidable) conditions:

$$\text{QR}' = \text{QR} \cap \left\{ (a, N) \in \mathbb{Z}_{>0}^2 \mid N \text{ is odd and } \gcd(a, N) = 1 \right\}.$$

A problem with two different p-verifiers

Consider the following language of quadratic residues (with inputs in binary):

$$\text{QR} = \left\{ (a, N) \in \mathbb{Z}_{>0}^2 \mid \exists x \in \mathbb{N} \ a \equiv_N x^2 \right\},$$

where $a \equiv_m b$ means that m divides $a - b$.

For simplicity, we add the following (polynomial-time decidable) conditions:

$$\text{QR}' = \text{QR} \cap \left\{ (a, N) \in \mathbb{Z}_{>0}^2 \mid N \text{ is odd and } \gcd(a, N) = 1 \right\}.$$

A natural p-verifier for QR' receives an $x \in \{0, 1, \dots, N-1\}$ and checks whether $x^2 \equiv_N a$.

Another witness: factoring N into primes

Instead of searching for a square root of a , we aim to find an y that encodes the prime factorization of N :

$$y = \langle (p_1, e_1), \dots, (p_t, e_t) \rangle.$$

Another witness: factoring N into primes

Instead of searching for a square root of a , we aim to find an y that encodes the prime factorization of N :

$$y = \langle (p_1, e_1), \dots, (p_t, e_t) \rangle.$$

The intended meaning is

$$N = \prod_{i=1}^t p_i^{e_i},$$

where $p_1, \dots, p_t$ are distinct odd primes and $e_1, \dots, e_t \in \mathbb{Z}_{>0}$.

Another witness: factoring N into primes

Instead of searching for a square root of a , we aim to find an y that encodes the prime factorization of N :

$$y = \langle (p_1, e_1), \dots, (p_t, e_t) \rangle.$$

The intended meaning is

$$N = \prod_{i=1}^t p_i^{e_i},$$

where $p_1, \dots, p_t$ are distinct odd primes and $e_1, \dots, e_t \in \mathbb{Z}_{>0}$.

Thanks to the AKS primality test, we can check in $\text{poly}(\log N)$ time whether y is actually the prime factorization of N .

Number theory to the rescue

Using the Chinese remainder theorem

a is a square modulo N iff it is a square modulo $p_i^{e_i}$ for each $i \in \{1, \dots, t\}$.

Number theory to the rescue

Using the Chinese remainder theorem

a is a square modulo N iff it is a square modulo $p_i^{e_i}$ for each $i \in \{1, \dots, t\}$.

Using Hensel lifting

For an odd prime p such that $\gcd(a, p) = 1$, a is a square modulo p^e iff it is a square modulo p .

Number theory to the rescue

Using the Chinese remainder theorem

a is a square modulo N iff it is a square modulo $p_i^{e_i}$ for each $i \in \{1, \dots, t\}$.

Using Hensel lifting

For an odd prime p such that $\gcd(a, p) = 1$, a is a square modulo p^e iff it is a square modulo p .

Using Euler's criterion

For an odd prime p such that $\gcd(a, p) = 1$, a is a square modulo p iff

$$a^{(p-1)/2} \equiv_p 1.$$

Number theory to the rescue

Using the Chinese remainder theorem

a is a square modulo N iff it is a square modulo $p_i^{e_i}$ for each $i \in \{1, \dots, t\}$.

Using Hensel lifting

For an odd prime p such that $\gcd(a, p) = 1$, a is a square modulo p^e iff it is a square modulo p .

Using Euler's criterion

For an odd prime p such that $\gcd(a, p) = 1$, a is a square modulo p iff

$$a^{(p-1)/2} \equiv_p 1.$$

So, once we know the prime factorization of N , membership in QR' can be checked prime by prime using fast exponentiation.

This gives us a second p-verifier for QR' .

A verifier-independent view of search-to-decision

We say that $M^\square$ is a *polynomial-time oracle machine (POM)* if there exists a polynomial $p(\cdot)$ such that, for every $x \in \{0,1\}^*$ and every language $A \subseteq \{0,1\}^*$, the running time of $M^A(x)$ is at most $p(|x|)$.

A verifier-independent view of search-to-decision

We say that $M^\square$ is a *polynomial-time oracle machine (POM)* if there exists a polynomial $p(\cdot)$ such that, for every $x \in \{0,1\}^*$ and every language $A \subseteq \{0,1\}^*$, the running time of $M^A(x)$ is at most $p(|x|)$.

We will say that $L \in \text{NP}$ is *search-to-decision reducible* if there exists **some** p-verifier V for L and POM $M^\square$ such that

$$\forall x \in L \quad V(x, M^L(x)) = \text{True}.$$

A verifier-independent view of search-to-decision

We say that $M^\square$ is a *polynomial-time oracle machine (POM)* if there exists a polynomial $p(\cdot)$ such that, for every $x \in \{0,1\}^*$ and every language $A \subseteq \{0,1\}^*$, the running time of $M^A(x)$ is at most $p(|x|)$.

We will say that $L \in \text{NP}$ is *search-to-decision reducible* if there exists **some** p-verifier V for L and POM $M^\square$ such that

$$\forall x \in L \quad V(x, M^L(x)) = \text{True}.$$

This is a very general definition. Note that proving that some $L \in \text{NP}$ is not search-to-decision reducible implies that all possible search problems associated to L are harder than the corresponding decision problem.

All NP-complete problems are search-to-decision reducible

Recall that $L \in \text{NP}$ is said to be NP-complete under Cook reductions if $\text{NP} \subseteq P^L$. This is a weaker property than the usual NP-completeness.

All NP-complete problems are search-to-decision reducible

Recall that $L \in \text{NP}$ is said to be NP-complete under Cook reductions if $\text{NP} \subseteq P^L$. This is a weaker property than the usual NP-completeness.

Let $L \in \text{NP}$ be NP-complete under Cook reductions. Then L is search-to-decision reducible.

All NP-complete problems are search-to-decision reducible

Recall that $L \in \text{NP}$ is said to be NP-complete under Cook reductions if $\text{NP} \subseteq P^L$. This is a weaker property than the usual NP-completeness.

Let $L \in \text{NP}$ be NP-complete under Cook reductions. Then L is search-to-decision reducible.

Proof.

- Let V be a p-verifier for L and let (f, g) be a Levin reduction from (L, V) to $(\text{SAT}, V_{\text{eval}})$.

All NP-complete problems are search-to-decision reducible

Recall that $L \in \text{NP}$ is said to be NP-complete under Cook reductions if $\text{NP} \subseteq P^L$. This is a weaker property than the usual NP-completeness.

Let $L \in \text{NP}$ be NP-complete under Cook reductions. Then L is search-to-decision reducible.

Proof.

- Let V be a p-verifier for L and let (f, g) be a Levin reduction from (L, V) to $(\text{SAT}, V_{\text{eval}})$.
- We know that there exists a POM $M_1^\square$ such that $V_{\text{eval}}(f(x), M_1^{\text{SAT}}(f(x))) = \text{True}$ for all $x \in L$.

All NP-complete problems are search-to-decision reducible

Recall that $L \in \text{NP}$ is said to be NP-complete under Cook reductions if $\text{NP} \subseteq P^L$. This is a weaker property than the usual NP-completeness.

Let $L \in \text{NP}$ be NP-complete under Cook reductions. Then L is search-to-decision reducible.

Proof.

- Let V be a p-verifier for L and let (f, g) be a Levin reduction from (L, V) to $(\text{SAT}, V_{\text{eval}})$.
- We know that there exists a POM $M_1^\square$ such that $V_{\text{eval}}(f(x), M_1^{\text{SAT}}(f(x))) = \text{True}$ for all $x \in L$.
- Because $\text{SAT} \in P^L$, we can simulate every SAT oracle query made by M_1 using an L-oracle. Hence there exists another POM $M_2^\square$ such that $V_{\text{eval}}(f(x), M_2^L(f(x))) = \text{True}$ for all $x \in L$.

All NP-complete problems are search-to-decision reducible

Recall that $L \in \text{NP}$ is said to be NP-complete under Cook reductions if $\text{NP} \subseteq P^L$. This is a weaker property than the usual NP-completeness.

Let $L \in \text{NP}$ be NP-complete under Cook reductions. Then L is search-to-decision reducible.

Proof.

- Let V be a p-verifier for L and let (f, g) be a Levin reduction from (L, V) to $(\text{SAT}, V_{\text{eval}})$.
- We know that there exists a POM $M_1^\square$ such that $V_{\text{eval}}(f(x), M_1^{\text{SAT}}(f(x))) = \text{True}$ for all $x \in L$.
- Because $\text{SAT} \in P^L$, we can simulate every SAT oracle query made by M_1 using an L-oracle. Hence there exists another POM $M_2^\square$ such that $V_{\text{eval}}(f(x), M_2^L(f(x))) = \text{True}$ for all $x \in L$.
- Define a third POM $M_3^\square$ such that $M_3^\square(x) = g(x, M_2^\square(f(x)))$ and note that $V(x, M_3^L(x)) = \text{True}$ for all $x \in L$.



Languages that are not search-to-decision reducible

Consider the following complexity classes:

$$EE_{BG} = DTIME \left(2^{2^{n+O(1)}} \right) \quad , \quad NEE_{BG} = NTIME \left(2^{2^{n+O(1)}} \right) .$$

Warning: These are different from the following, more standard classes:

$$EE = DTIME \left(2^{2^{O(n)}} \right) \quad , \quad NEE = NTIME \left(2^{2^{O(n)}} \right) .$$

Languages that are not search-to-decision reducible

Consider the following complexity classes:

$$EE_{BG} = DTIME\left(2^{2^{n+O(1)}}\right) \quad , \quad NEE_{BG} = NTIME\left(2^{2^{n+O(1)}}\right) .$$

Warning: These are different from the following, more standard classes:

$$EE = DTIME\left(2^{2^{O(n)}}\right) \quad , \quad NEE = NTIME\left(2^{2^{O(n)}}\right) .$$

Theorem (Bellare and Goldwasser, 1994)

If $EE_{BG} \neq NEE_{BG}$, then there exists a language in NP which is not search-to-decision reducible.

Uniformly log-sparse languages

Given any language $L \subseteq \{0, 1\}^*$, define the function $\mu_L: \mathbb{N} \rightarrow \mathbb{N}$ by

$$\mu_L(n) = \left| L \cap \{0, 1\}^{\leq n} \right|.$$

We say that L is *log-sparse* if $\mu_L(n) = O(\log n)$.

Uniformly log-sparse languages

Given any language $L \subseteq \{0, 1\}^*$, define the function $\mu_L: \mathbb{N} \rightarrow \mathbb{N}$ by

$$\mu_L(n) = \left| L \cap \{0, 1\}^{\leq n} \right|.$$

We say that L is *log-sparse* if $\mu_L(n) = O(\log n)$.

We say that a language $C \subseteq \{0, 1\}^*$ is a *candidate selector* for another language $L \subseteq \{0, 1\}^*$ if $C \in \mathsf{P}$ and $L \subseteq C$.

Uniformly log-sparse languages

Given any language $L \subseteq \{0, 1\}^*$, define the function $\mu_L: \mathbb{N} \rightarrow \mathbb{N}$ by

$$\mu_L(n) = \left| L \cap \{0, 1\}^{\leq n} \right|.$$

We say that L is *log-sparse* if $\mu_L(n) = O(\log n)$.

We say that a language $C \subseteq \{0, 1\}^*$ is a *candidate selector* for another language $L \subseteq \{0, 1\}^*$ if $C \in \mathsf{P}$ and $L \subseteq C$.

We say that $L \subseteq \{0, 1\}^*$ is *uniformly log-sparse* if it has a log-sparse candidate selector.

The padding argument

If $EE_{BG} \neq NEE_{BG}$, then there exists a uniformly log-sparse language $L \in NP \setminus P$.

The padding argument

If $EE_{BG} \neq NEE_{BG}$, then there exists a uniformly log-sparse language $L \in NP \setminus P$.

Proof.

Take an $L' \in NEE_{BG} \setminus EE_{BG}$. Consider $g(y) := 2^{2^{|y|}}$ and define

$$L = \left\{ y \cdot 0^{g(y)-|y|} \mid y \in L' \right\}.$$

As usual, notice that $L \in NP \setminus P$.

The padding argument

If $EE_{BG} \neq NEE_{BG}$, then there exists a uniformly log-sparse language $L \in NP \setminus P$.

Proof.

Take an $L' \in NEE_{BG} \setminus EE_{BG}$. Consider $g(y) := 2^{2^{|y|}}$ and define

$$L = \left\{ y \cdot 0^{g(y)-|y|} \mid y \in L' \right\}.$$

As usual, notice that $L \in NP \setminus P$. Now define the following algorithm A :

On input $x \in \{0,1\}^*$, computes $m \in \mathbb{N}$ such that $|x| = 2^{2^m}$ (if m does not exist, returns 0). Then A outputs 1 if the last $|x| - m$ characters of x are equal to 0, and outputs 0 otherwise.

The padding argument

If $EE_{BG} \neq NEE_{BG}$, then there exists a uniformly log-sparse language $L \in NP \setminus P$.

Proof.

Take an $L' \in NEE_{BG} \setminus EE_{BG}$. Consider $g(y) := 2^{2^{|y|}}$ and define

$$L = \left\{ y \cdot 0^{g(y)-|y|} \mid y \in L' \right\}.$$

As usual, notice that $L \in NP \setminus P$. Now define the following algorithm A :

On input $x \in \{0, 1\}^*$, computes $m \in \mathbb{N}$ such that $|x| = 2^{2^m}$ (if m does not exist, returns 0). Then A outputs 1 if the last $|x| - m$ characters of x are equal to 0, and outputs 0 otherwise.

Then $\{x \in \{0, 1\}^* \mid A(x) = 1\}$ is a candidate selector for L . Log-sparsity holds because, up to length $n = 2^{2^m}$, it contains at most $\sum_{k=0}^m 2^k = O(\log(n))$ strings. □

The crucial lemma

Suppose $L \in \text{NP}$ is a uniformly log-sparse language that is search-to-decision reducible. Then $L \in \text{P}$.

The crucial lemma

Suppose $L \in \text{NP}$ is a uniformly log-sparse language that is search-to-decision reducible. Then $L \in \text{P}$.

Proof (sketch).

- Let V be a p-verifier for L , $M^\square$ a POM such that $V(x, M^L(x)) = \text{True}$ for all $x \in L$, and C a candidate selector for L .

The crucial lemma

Suppose $L \in \text{NP}$ is a uniformly log-sparse language that is search-to-decision reducible. Then $L \in \text{P}$.

Proof (sketch).

- Let V be a p-verifier for L , $M^\square$ a POM such that $V(x, M^L(x)) = \text{True}$ for all $x \in L$, and C a candidate selector for L .
- Note that there exists a constant $c > 0$ such that $M^\square$ on input x runs for at most $|x|^c$ steps, regardless of the oracle. In particular, note that all oracle queries will have size bounded by $|x|^c$.

The crucial lemma

Suppose $L \in \text{NP}$ is a uniformly log-sparse language that is search-to-decision reducible. Then $L \in \text{P}$.

Proof (sketch).

- Let V be a p-verifier for L , $M^\square$ a POM such that $V(x, M^L(x)) = \text{True}$ for all $x \in L$, and C a candidate selector for L .
- Note that there exists a constant $c > 0$ such that $M^\square$ on input x runs for at most $|x|^c$ steps, regardless of the oracle. In particular, note that all oracle queries will have size bounded by $|x|^c$.
- For an input $x \in \{0, 1\}^*$, we iterate over all languages A contained in $C \cap \{0, 1\}^{\leq |x|^c}$. There are $2^{O(\log(|x|^c))} = \text{poly}(|x|)$ many such A 's.

The crucial lemma

Suppose $L \in \text{NP}$ is a uniformly log-sparse language that is search-to-decision reducible. Then $L \in \text{P}$.

Proof (sketch).

- Let V be a p-verifier for L , $M^\square$ a POM such that $V(x, M^L(x)) = \text{True}$ for all $x \in L$, and C a candidate selector for L .
- Note that there exists a constant $c > 0$ such that $M^\square$ on input x runs for at most $|x|^c$ steps, regardless of the oracle. In particular, note that all oracle queries will have size bounded by $|x|^c$.
- For an input $x \in \{0, 1\}^*$, we iterate over all languages A contained in $C \cap \{0, 1\}^{\leq |x|^c}$. There are $2^{O(\log(|x|^c))} = \text{poly}(|x|)$ many such A 's.
- For each A , we **simulate** $M^A(x)$, answering FALSE for every oracle query outside of C (recall that $C \in \text{P}$). We save the output as w_A .

The crucial lemma

Suppose $L \in \text{NP}$ is a uniformly log-sparse language that is search-to-decision reducible. Then $L \in \text{P}$.

Proof (sketch).

- Let V be a p-verifier for L , $M^\square$ a POM such that $V(x, M^L(x)) = \text{True}$ for all $x \in L$, and C a candidate selector for L .
- Note that there exists a constant $c > 0$ such that $M^\square$ on input x runs for at most $|x|^c$ steps, regardless of the oracle. In particular, note that all oracle queries will have size bounded by $|x|^c$.
- For an input $x \in \{0, 1\}^*$, we iterate over all languages A contained in $C \cap \{0, 1\}^{\leq |x|^c}$. There are $2^{O(\log(|x|^c))} = \text{poly}(|x|)$ many such A 's.
- For each A , we **simulate** $M^A(x)$, answering FALSE for every oracle query outside of C (recall that $C \in \text{P}$). We save the output as w_A .
- We accept x iff there is an A such that $V(x, w_A) = \text{True}$.



Concluding the proof

Theorem (Bellare and Goldwasser, 1994)

If $EE_{BG} \neq NEE_{BG}$, then there exists a language in NP which is not search-to-decision reducible.

Concluding the proof

Theorem (Bellare and Goldwasser, 1994)

If $EE_{BG} \neq NEE_{BG}$, then there exists a language in NP which is not search-to-decision reducible.

Proof.

Suppose that $EE_{BG} \neq NEE_{BG}$. Then there exists a uniformly log-sparse language $L \in NP \setminus P$. If L were search-to-decision reducible, then by the previous lemma we would have $L \in P$, a contradiction. $\square$

Concluding the proof

Theorem (Bellare and Goldwasser, 1994)

If $EE_{BG} \neq NEE_{BG}$, then there exists a language in NP which is not search-to-decision reducible.

Proof.

Suppose that $EE_{BG} \neq NEE_{BG}$. Then there exists a uniformly log-sparse language $L \in NP \setminus P$. If L were search-to-decision reducible, then by the previous lemma we would have $L \in P$, a contradiction. $\square$

Corollary

If $EE_{BG} \neq NEE_{BG}$, then there is a language in $NP \setminus P$ which is not NP-complete under Cook reductions.

References



Bellare, Mihir and Shafi Goldwasser (1994). “The complexity of decision versus search”. In: *SIAM J. Comput.* 23.1, pp. 97–119. ISSN: 0097-5397. DOI: 10.1137/S0097539792228289. URL: <https://doi-org.pucdechile.idm.oclc.org/10.1137/S0097539792228289>.